# SEN 304 – SOFTWARE TESTING AND QUALITY ASSURANCE

# GROUP REPORT

# TEAM MEMBERS


| NAME | EMAIL | STUDENT ID | MATRIC NO |
|---|---|---|---|
| Johnny Miracle Aniekan | miracle.johnny@miva.edu.ng | 30021254 | 2023/A/SENG/0111 |
| Jephthah Enobakhare | Jephthah.enobakhare@miva.edu.ng | 30055142 | 2023/C/SENG/0366 |
| Bright Oghor | Bright.oghor@miva.edu.ng | 30074665 | 2023/C/SENG/0367 |
| Abdulrahman Adewale Adegeye | abdulrahman.adegeye@miva.edu.ng | 30015471 | 2023/C/SENG/0305 |
| | | | |


# 1. PROJECT OVERVIEW

The **MIVA Level 300 Result Calculator System** is a Python-based application developed to compute student academic performance.

The system performs the following operations:

- Accepts student scores
- Calculates the average score
- Assigns grades based on defined thresholds
- Displays pass/fail status

## Part A: Debugging Implementation

In Part A, the system was intentionally developed with multiple types of errors, including:

- Syntax errors
- Runtime errors
- Logical errors

These errors were identified and resolved using debugging tools within an IDE. Techniques such as:

- Setting breakpoints
- Stepping through code (Step Into and Step Over)
- Inspecting variables
- Using watch expressions

were applied to trace and fix the issues. The goal was to demonstrate practical debugging skills and improve code reliability.

## Part B: Unit Testing with PyTest

In Part B, the corrected system was tested using the **PyTest framework** to ensure correctness and robustness.

The testing process included:

- **Black-box testing techniques**:
    - Equivalence Partitioning
    - Boundary Value Analysis

- **White-box testing techniques**:
    - Statement Coverage
    - Branch Coverage
    - Path Coverage

Comprehensive test cases were written to cover:

- Normal inputs
- Boundary conditions
- Edge cases (e.g., empty input)
- Error handling scenarios

The system achieved **100% test coverage**, confirming that all parts of the code were executed and validated during testing.

Overall, this project demonstrates the complete software testing process, from debugging faulty code to validating the final system using structured testing methodologies.

# 2. TEAM MEMBERS AND TASK DISTRIBUTION

| Name | Role | Responsibility |
|---|---|---|
| Bright Oghor | Developer / Tester | Designed the system, implemented core logic (`calculate_average`, `get_grade`), identified and fixed major bugs (syntax, TypeError, NameError), configured PyTest, resolved import and coverage issues, achieved 100% test coverage, and compiled the final report |
| Johnny Miracle Aniekan | | Set breakpoints in Visual Studio Code, stepped through execution using F10/F11, inspected variables (`scores[i]`, `total`), identified runtime TypeError caused by string input, and validated fixes during debugging sessions |
| | | Wrote unit tests in `test_project.py`, implemented equivalence partitioning and boundary value tests, |

| | | |
|---|---|---|
| | | created tests for grade boundaries (45, 50, 60, 70), and ensured all functions were properly tested |
| | | Verified correctness of outputs, confirmed bug fixes (syntax, runtime, logical), validated pass/fail logic, and ensured consistency between expected and actual results |
| | | Executed coverage analysis using `pytest --cov=project`, interpreted coverage reports, confirmed 100% statement coverage, and ensured no untested code paths remained |
| | | Designed and tested edge cases (empty list, invalid inputs like `"abc"`), validated exception handling using `pytest.raises`, and ensured program stability under invalid conditions |
| | | Organized screenshots (debugging, errors, test runs, coverage), structured debugging and testing reports, formatted the final submission document, and ensured all requirements were met |
| | | |

# 3. DEBUGGING LOGS AND BUG-FIX REPORTS

## Overview

The program initially contained multiple errors, including syntax, runtime, and logical errors. These were identified and resolved using debugging tools in Visual Studio Code.

## Bug 1: Syntax Error

**Location:**

```
if average > 50
```

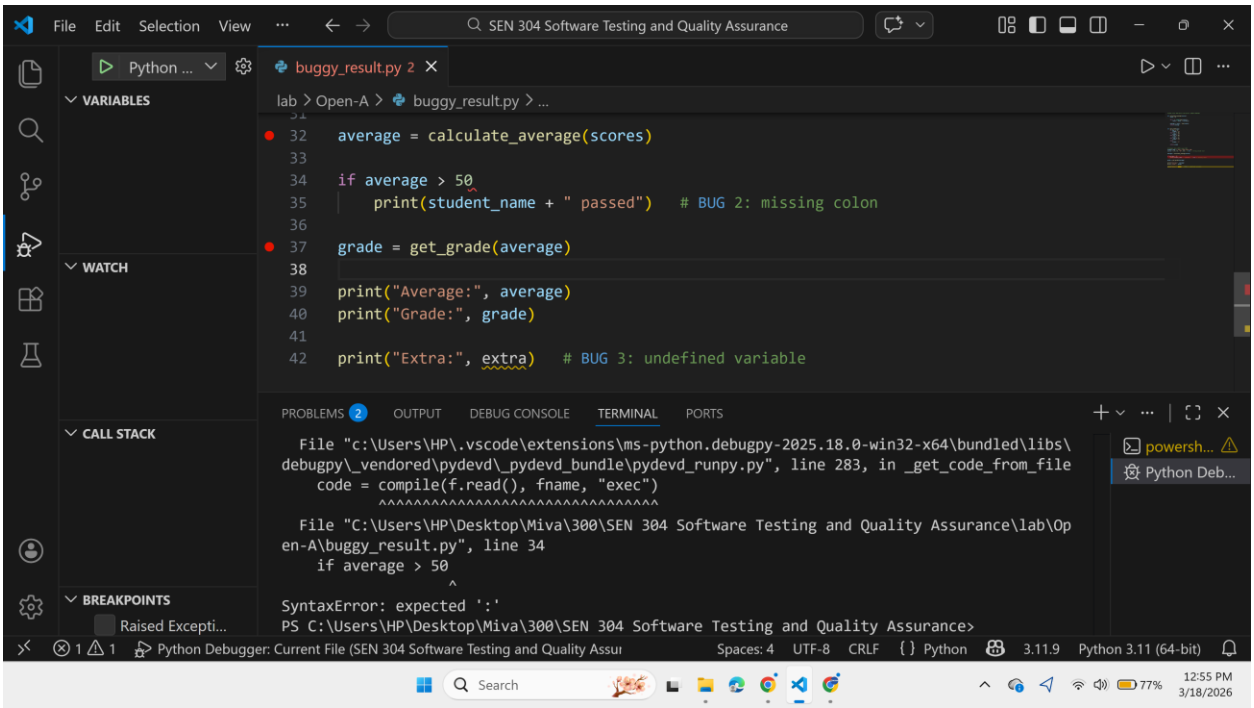
**Issue:**

Missing colon : caused program failure.

**Fix:**

```
if average > 50:
```

**Screenshot:**




## Bug 2: Runtime Error (Type Error)

**Location:**

```
scores = [70, 65, "80", 90]
```

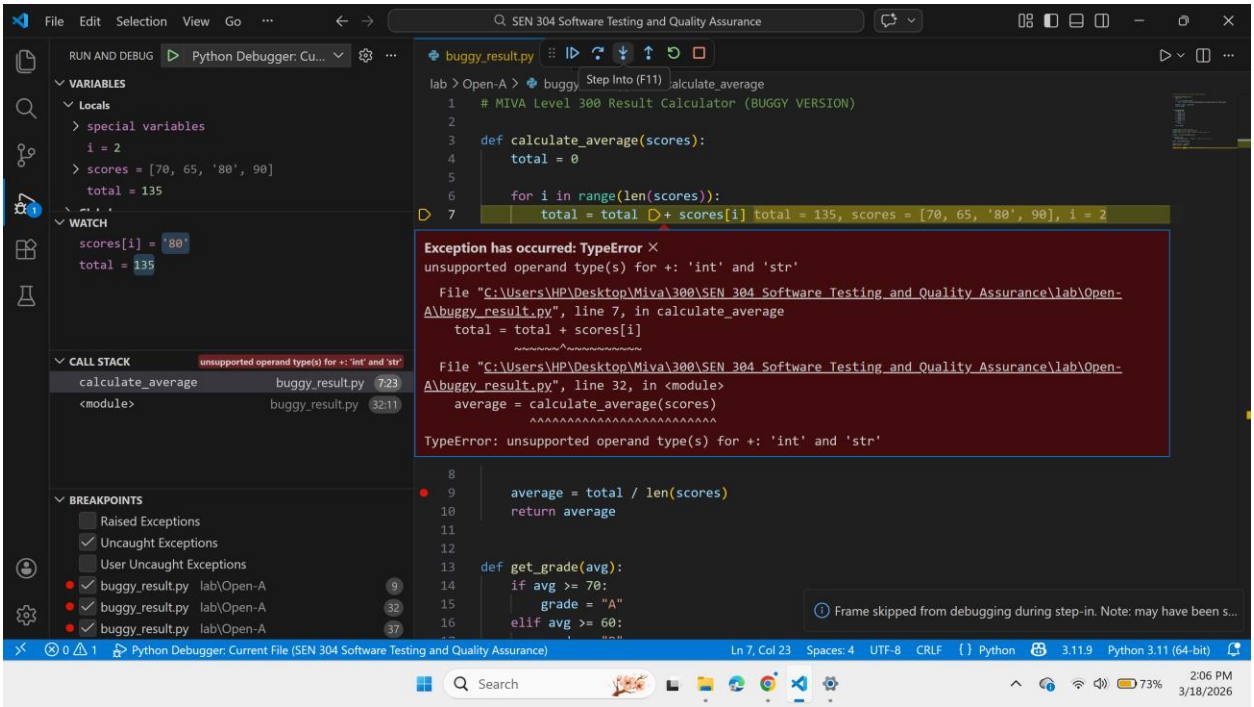
**Issue:**

String value caused arithmetic failure.

**Fix:**

```
total = total + float(scores[i])
```

**Screenshot:**




## Bug 3: NameError

**Location:**

```
print("Extra:", extra)
```

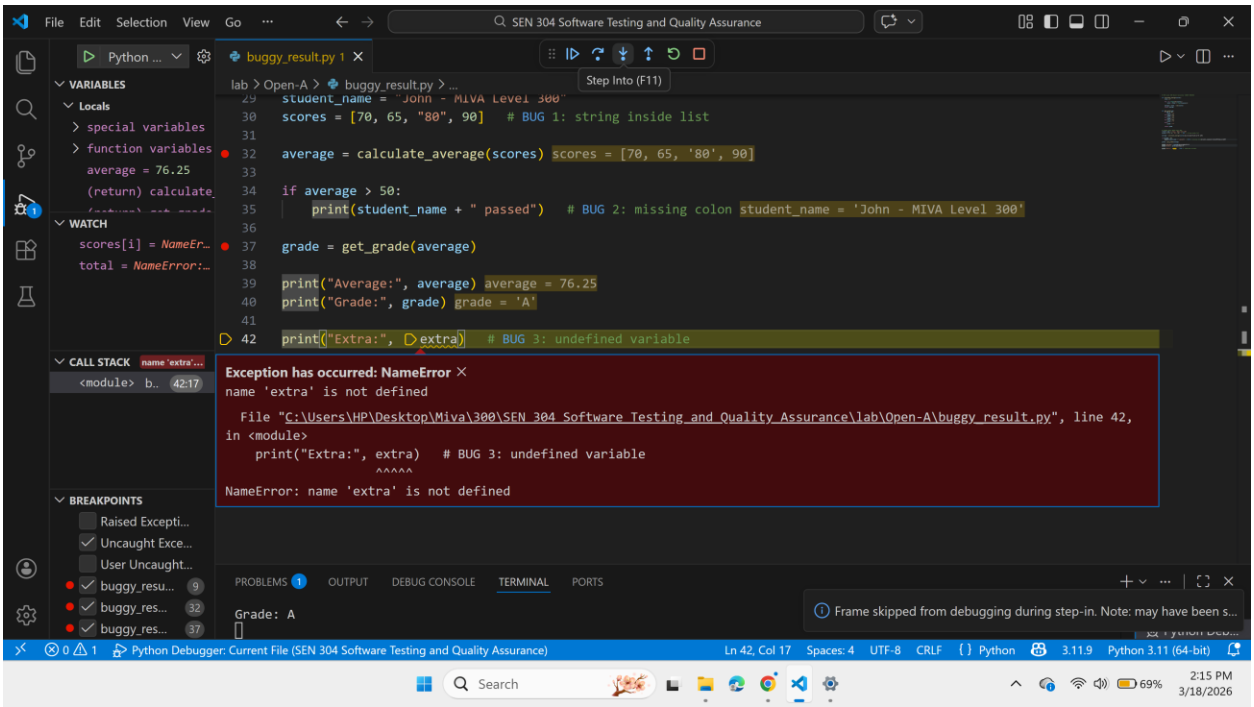
**Issue:**

Undefined variable `extra`.

**Fix:**

Removed the line.

**Screenshot:**




# Bug 4: Logical Issue

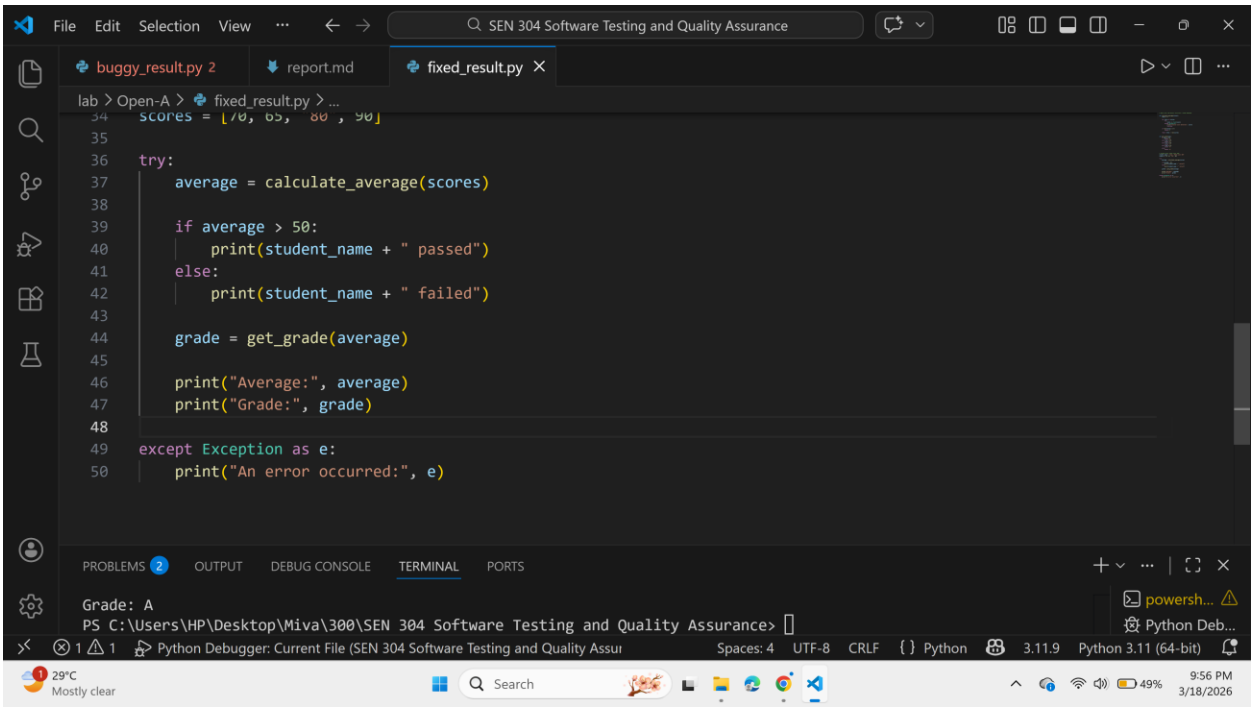
**Issue:**

No validation for input values.

**Fix:**

Converted inputs to numeric values safely.

**Screenshot:**




# 4. UNIT TESTING STRATEGY AND SUMMARY

## Testing Approach

Testing was conducted using the **PyTest framework** to validate all functionalities of the system.

### 4.1 Black-Box Testing

#### *Equivalence Partitioning*

- Valid inputs → `[70, 80, 90]`
- Mixed inputs → `[70, "80", 90]`
- Invalid inputs → `[10, "abc", 20]`

#### *Boundary Value Analysis*

- Grade thresholds tested: `45, 50, 60, 70`
- Minimum: `0`
- Maximum: `100`

### 4.2 White-Box Testing

#### *Statement Coverage*

- All lines of code executed

#### *Branch Coverage*

- All grade conditions tested (A–F)

#### *Path Coverage*

- Multiple execution paths verified

## 5. TEST COVERAGE RESULT

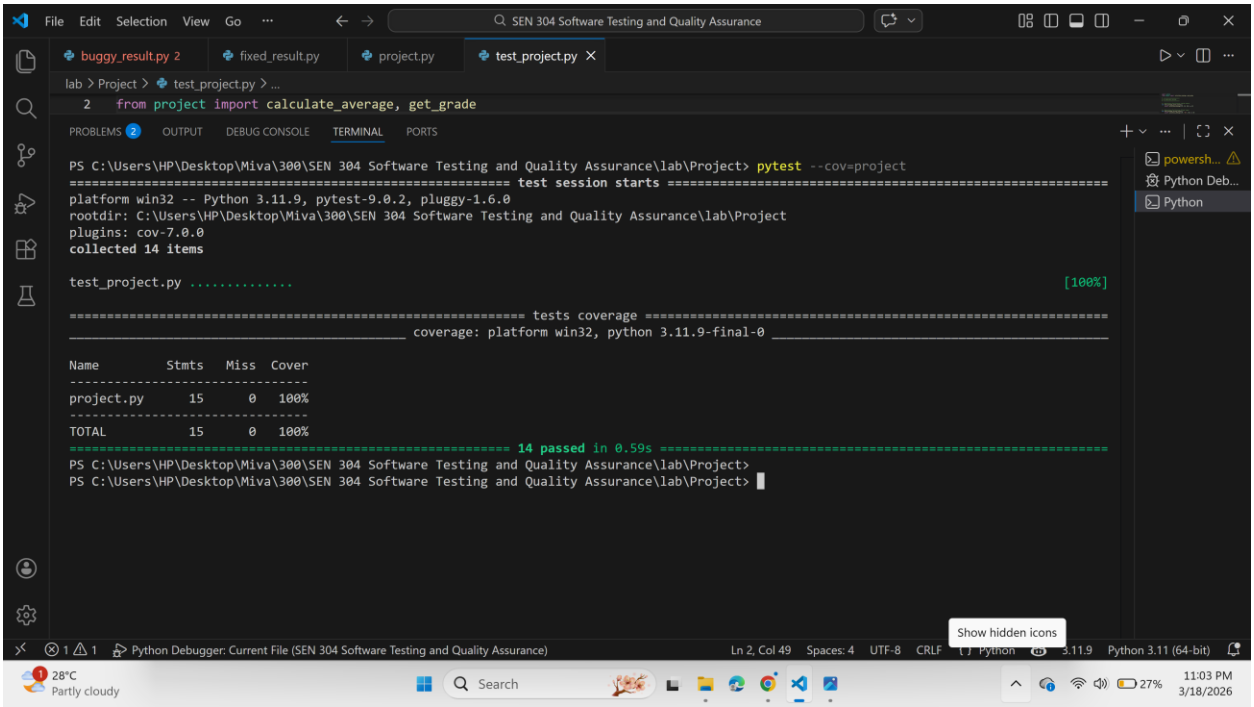
**Command Used:**

```
pytest --cov=project
```

**Result:**

```
100% Coverage Achieved
```

## Screenshot:




# 6. TEST EXECUTION RESULT

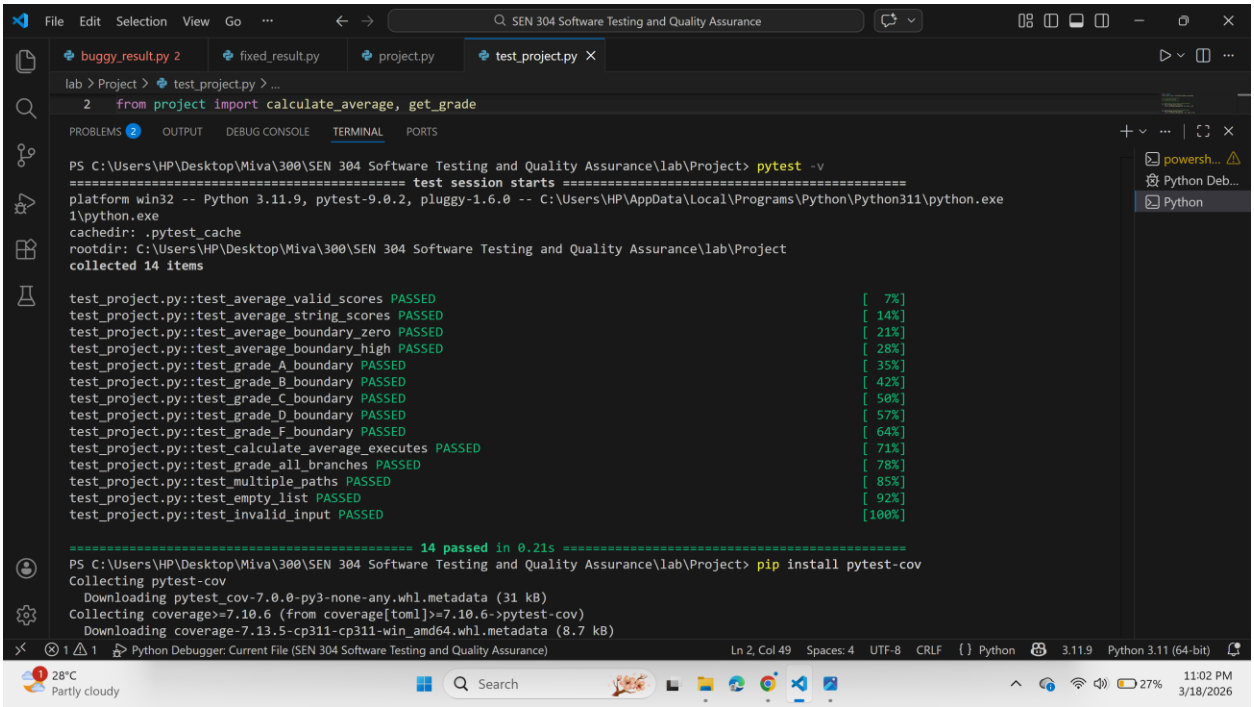
**Command Used:**

```
pytest -v
```

**Result:**

- 14 tests collected
- 14 tests passed
- 0 failures

## Screenshot:




# 7. CHALLENGES FACED

## 1. Import Error

- Incorrect relative import
- Fixed using absolute import

## 2. Coverage Not Detected

- Wrong module name used
- Fixed with:

```
pytest --cov=project
```

## 3. Handling Invalid Inputs

- Program crashed with non-numeric values

- Fixed using proper test cases and conversions

# 8. CONCLUSION

This project demonstrated practical skills in:

- Debugging Python programs
- Using IDE tools effectively
- Writing unit tests with PyTest
- Achieving full test coverage

The system is now fully functional, tested, and reliable.